

Problem Description:

Any sequence A of size n is called **B-sequence** if:

$A_1 < A_2 < \dots < A_k > A_{k+1} > A_{k+2} > \dots > A_n$ where $1 \leq k \leq n$. That is, a sequence which is initially strictly increasing and then strictly decreasing (the decreasing part may or may not be there).

All elements in A except the maximum element comes atmost twice (once in increasing part and once in decreasing part) and maximum element comes exactly once.

All elements coming in decreasing part of sequence should have come once in the increasing part of sequence.

You are given a B-sequence S and Q operations. For each operation, you are given a value **val**. You have to insert **val** in S if and only if after insertion, S still remains a B-sequence .

After each operation, print the size of S . After all the operations, print the sequence S .

Hint: Think of using some data structure to support insertion of elements in complexity better than linear.

Input Constraints:

$$1 \leq N \leq 10^5$$

$$1 \leq S_i \leq 10^9$$

$$1 \leq Q \leq 10^5$$

$$1 \leq val \leq 10^9$$

Given sequence S is a **B-sequence**,

Input Format:

First line consists of an integer N , denoting size of S .

Second line consists of N space separated integers, denoting elements of S .

Next line consists of an integer **Q**, denoting number of operations.

Each of the following **Q** lines consists of an integer **val**.

Output Format:

After each operation, print the size of **S** in a new line.

After all operations, print the sequence **S**.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

4
1 2 5 2
4
5
1
3
2

EXPECTED OUTPUT

4
5
6
6
1 2 3 5 2 1

Test Case 2

INPUT (STDIN)

5
3 4 5 1 3
3
6
2
3
1

EXPECTED OUTPUT

6
7
7
1 2 3 4 5 6 3

▼ Mandatory Test Cases

Test Case 1

KEYWORD

for(i=0;i<N;i++)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

8

Test Case 2

TOKEN COUNT

275

Test Case 3

NLOC

39

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 #define HASH_SIZE 1000003
6
7 typedef struct {
8     int val;
9     int count;
10    int used;
11} HashEntry;
12
13 HashEntry hash_table[HASH_SIZE];
14
15 int find_slot(int val) {
16     unsigned int h = ((unsigned int)val * 2654435761U) % HASH_SIZE;
17     while (hash_table[h].used && hash_table[h].val != val) {
18         h = (h + 1) % HASH_SIZE;
19     }
20     return h;
21 }
22
23 int get_count(int val) {
24     int slot = find_slot(val);
25     if (!hash_table[slot].used) return 0;
```

```

26     return hash_table[slot].count;
27 }
28
29 void set_count(int val, int count) {
30     int slot = find_slot(val);
31     hash_table[slot].val = val;
32     hash_table[slot].count = count;
33     hash_table[slot].used = 1;
34 }
35
36 int cmp_int(const void *a, const void *b) {
37     int x = *(const int*)a;
38     int y = *(const int*)b;
39     if (x < y) return -1;
40     if (x > y) return 1;
41     return 0;
42 }
43
44 int main() {
45     int N;

```

```

46     if (scanf("%d", &N) != 1) return 0;
47
48     int *init_s = (int*)malloc(N * sizeof(int));
49     int i;
50     for(i=0;i<N;i++) {
51         if (scanf("%d", &init_s[i]) != 1) init_s[i] = 0;
52     }
53
54     int *unique_vals = (int*)malloc(300005 * sizeof(int));
55     int unique_cnt = 0;
56     int current_size = N;
57     int max_val = init_s[0];
58
59     for (i = 0; i < N; i++) {
60         int v = init_s[i];
61         if (v > max_val) max_val = v;

```

main | CH.SC.U4CSE25225 | Tree27.c

Ln 122, Col 1 Spaces: 4 UTF-8 CRLF C

```

63         if (c == 0) {
64             unique_vals[unique_cnt++] = v;
65         }

```

```
66     set_count(v, c + 1);
67 }
68
69 int Q;
70 if (scanf("%d", &Q) != 1) Q = 0;
71
72 while (Q--) {
73     int val;
74     if (scanf("%d", &val) != 1) break;
75     if (val > max_val) {
76         int c = get_count(val);
77         if (c == 0) unique_vals[unique_cnt++] = val;
78         set_count(val, 1);
79         max_val = val;
80         current_size++;
81     } else if (val == max_val) {
82     } else {
83         int c = get_count(val);
84         if (c == 0) {
```

```

85         unique_vals[unique_cnt++] = val;
86         set_count(val, 1);
87         current_size++;
88     } else if (c == 1) {
89         set_count(val, 2);
90         current_size++;
91     }
92 }
93 printf("%d\n", current_size);
94 }
95
96 qsort(unique_vals, unique_cnt, sizeof(int), cmp_int);
97
98 int first = 1;
99 for (i = 0; i < unique_cnt; i++) {
100     int v = unique_vals[i];
101     if (get_count(v) >= 1) {
102         if (!first) printf(" ");
103         printf("%d", v);

```

```
104         first = 0;
105     }
106 }
107
108 for (i = unique_cnt - 1; i >= 0; i--) {
109     int v = unique_vals[i];
110     if (get_count(v) == 2 && v != max_val) {
111         if (!first) printf(" ");
112         printf("%d", v);
113         first = 0;
114     }
115 }
116 printf("\n");
117
118 free(init_s);
119 free(unique_vals);
120 return 0;
121 }
122
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

4

1 2 5 2

4

5

1

3

2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree27.exe

4

5

6

6

1 2 3 5 2 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

5

3 4 5 1 3

3

6

2

3

1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree27.exe

6

7

7

1 2 3 4 5 6 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>